

Data Exploration

Load raw data, visualize samples, class distribution, and raw histograms (RGB vs. grayscale).

```
# Cell 1.1: Import libraries & explore folder structure
import os
import cv2
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt

base_dir = '/kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images'
categories = ['VeryMildDemented', 'NonDemented', 'ModerateDemented', 'MildDemented']

# Folder counts
folder_counts = {}
for dirname, subdirs, filenames in os.walk(base_dir):
    folder_counts[dirname] = sum(1 for f in filenames if
f.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp')))

for folder, count in folder_counts.items():
    print(f"Folder: {folder}, Images: {count}")

Folder: /kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images, Images: 0
Folder: /kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images/ModerateDemented, Images: 10000
Folder: /kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images/NonDemented, Images: 12800
Folder: /kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images/VeryMildDemented, Images: 11200
Folder: /kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented/combined_images/MildDemented, Images: 10000

# Cell 1.2: Load raw images (RGB for exploration)
images_raw = []
labels_raw = []

for category in categories:
    path = os.path.join(base_dir, category)
    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name)
        try:
            img = cv2.imread(img_path)
            img = cv2.resize(img, (128, 128)) # Small for exploration
            images_raw.append(img)
            labels_raw.append(categories.index(category))
```

```

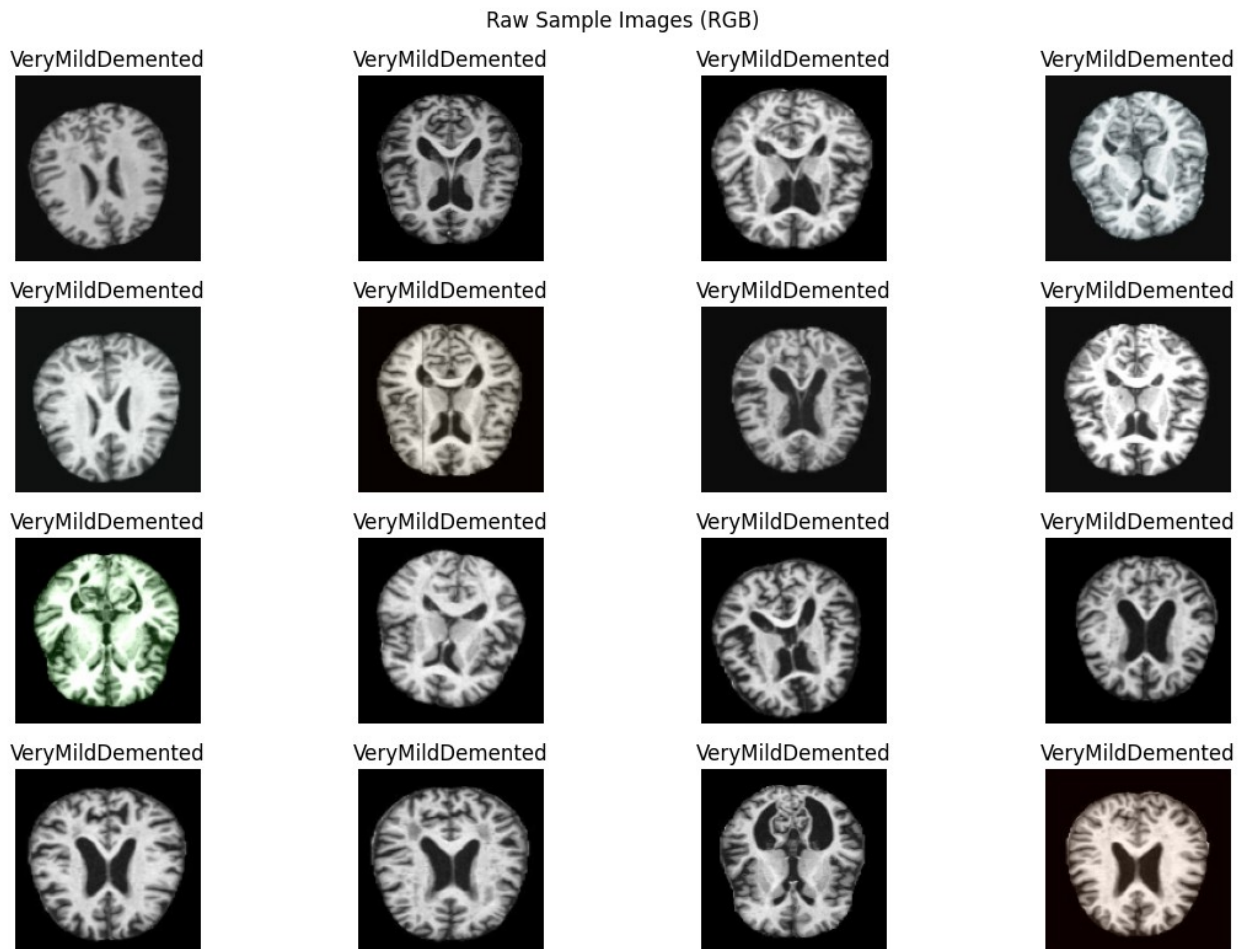
except Exception as e:
    print(f"Error: {e}")

images_raw = np.array(images_raw)
labels_raw = np.array(labels_raw)
print(f"Raw images shape: {images_raw.shape}")
print(f"Labels: {Counter(labels_raw)}")

Raw images shape: (44000, 128, 128, 3)
Labels: Counter({1: 12800, 0: 11200, 2: 10000, 3: 10000})

# Cell 1.3: Visualize raw samples
plt.figure(figsize=(12, 8))
for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(cv2.cvtColor(images_raw[i], cv2.COLOR_BGR2RGB))
    plt.title(f'{categories[labels_raw[i]]}')
    plt.axis('off')
plt.suptitle('Raw Sample Images (RGB)')
plt.tight_layout()
plt.show()

```

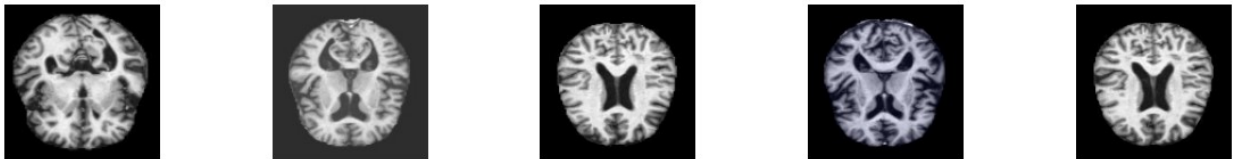


```

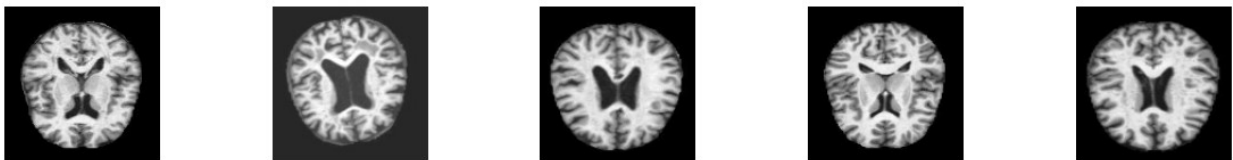
# Cell 1.4: Samples per class
for i, category in enumerate(categories):
    category_indices = np.where(labels_raw == i)[0]
    num_samples = min(5, len(category_indices))
    random_indices = np.random.choice(category_indices, num_samples,
    replace=False)
    plt.figure(figsize=(12, 2))
    plt.suptitle(f"Samples for {category}")
    for j, idx in enumerate(random_indices):
        plt.subplot(1, num_samples, j + 1)
        plt.imshow(cv2.cvtColor(images_raw[idx], cv2.COLOR_BGR2RGB))
        plt.axis('off')
    plt.tight_layout()
    plt.show()

```

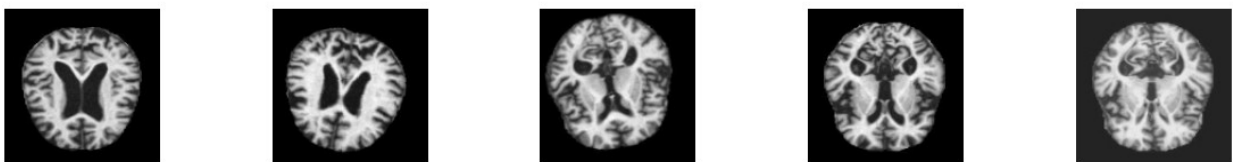
Samples for VeryMildDemented



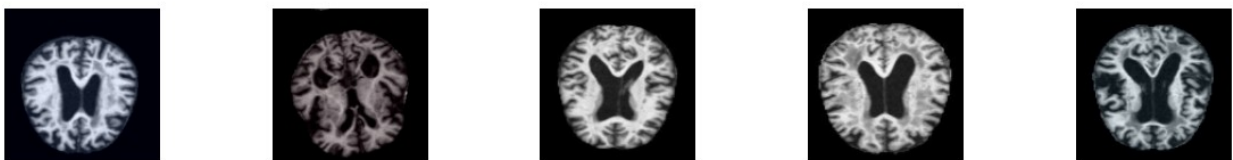
Samples for NonDemented



Samples for ModerateDemented



Samples for MildDemented



```

# Cell 1.5: Raw RGB histogram
sample_size = 1000
random_indices = np.random.choice(images_raw.shape[0], sample_size,
    replace=False)

```

```

sampled_images = images_raw[random_indices]

mean_pixels = np.mean(sampled_images, axis=(0, 1, 2))
std_pixels = np.std(sampled_images, axis=(0, 1, 2))
print(f"Mean RGB: {mean_pixels}")
print(f"Std RGB: {std_pixels}")

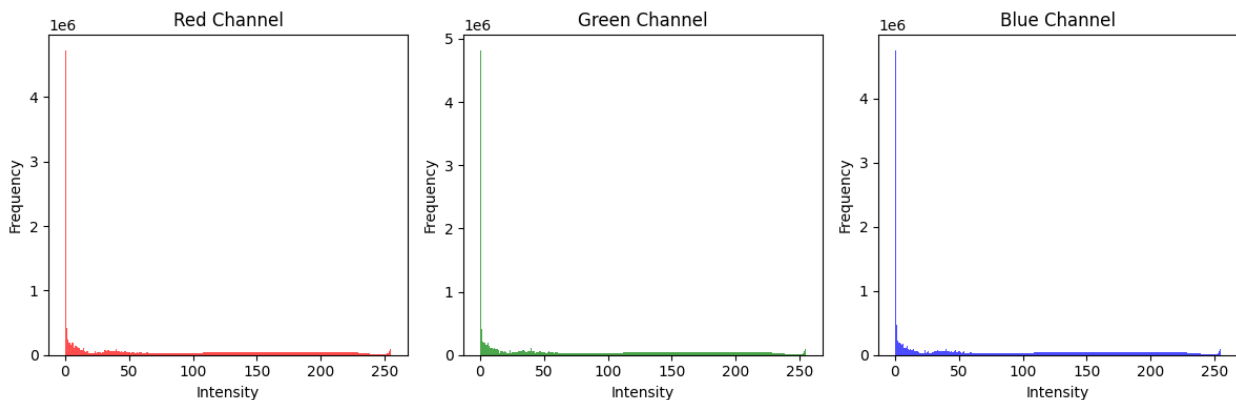
plt.figure(figsize=(12, 4))
for i, color in enumerate(['Red', 'Green', 'Blue']):
    plt.subplot(1, 3, i + 1)
    plt.hist(sampled_images[:, :, :, i].ravel(), bins=256,
color=color.lower(), alpha=0.7)
    plt.title(f'{color} Channel')
    plt.xlabel('Intensity')
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

```

```

Mean RGB: [74.35715375 74.18415723 74.33808002]
Std RGB: [81.62765948 81.57183995 81.63393219]

```



```

# Cell 1.6: Raw grayscale histogram (fixed: batch conversion)
# Use numpy for batch grayscale (avoids cv2 on 4D array)
weights = np.array([0.299, 0.587, 0.114]) # Luminance formula
images_gray = np.dot(images_raw, weights) # (N, H, W)

mean_gray = np.mean(images_gray, axis=(0, 1))
std_gray = np.std(images_gray, axis=(0, 1))
print(f"Mean Grayscale: {mean_gray}")
print(f"Std Grayscale: {std_gray}")

plt.figure(figsize=(8, 6))
plt.hist(images_gray.ravel(), bins=256, color='gray', alpha=0.7)
plt.title('Raw Grayscale Histogram')
plt.xlabel('Intensity')
plt.ylabel('Frequency')
plt.show()

```

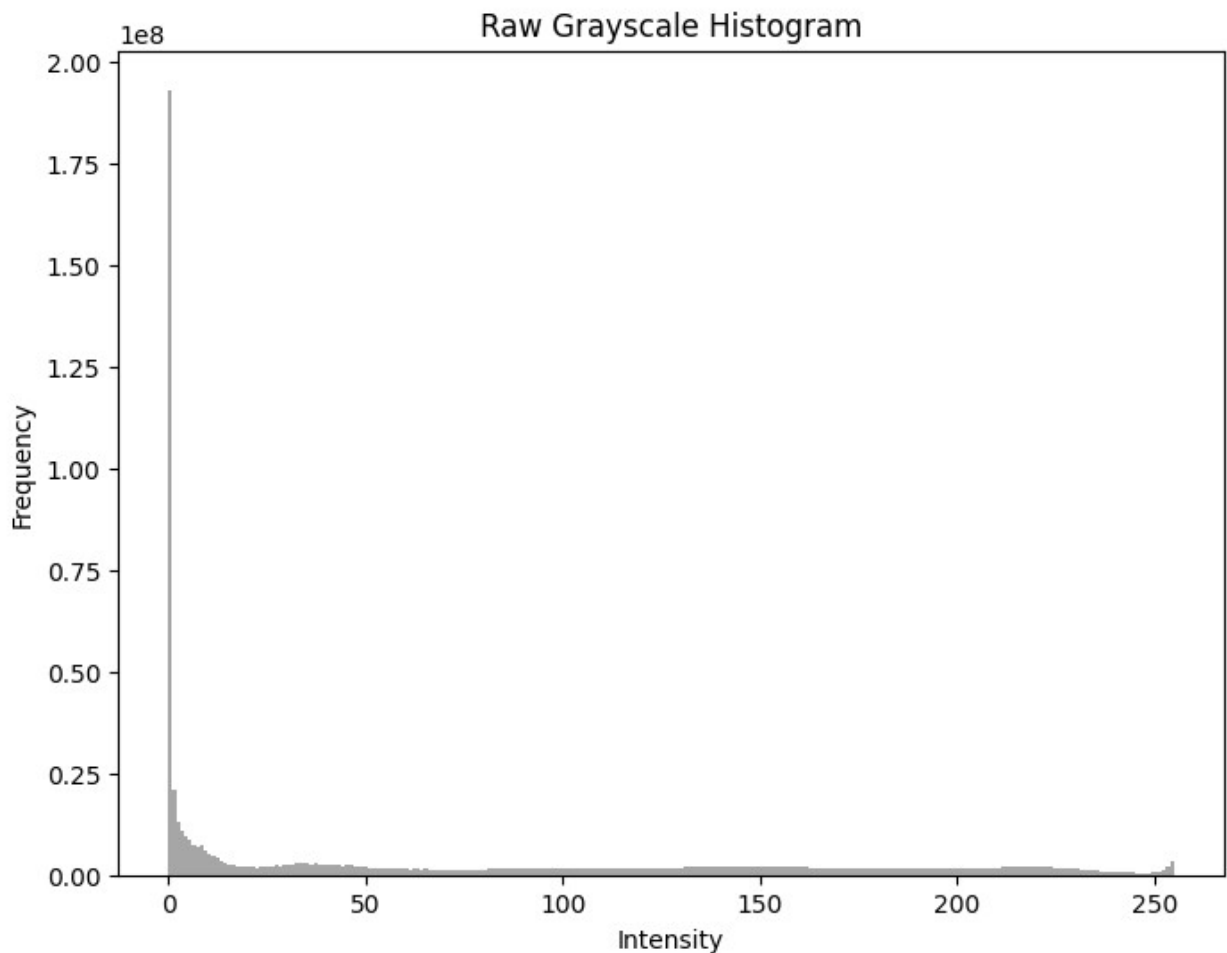
Mean Grayscale: [5.7599513 5.86938951 5.92297029 5.95902413
5.98040953

6.02896914	6.16616842	6.74263273	7.5834683	8.95542339
10.11046896	11.67872362	13.52807223	17.08681331	20.98222699
25.728907	30.87508588	35.87934335	40.81811487	45.86098812
50.49413201	54.77816954	59.65014562	64.3612301	68.5957218
72.72298456	76.813379	80.78265525	84.64286383	88.44404497
91.98678019	95.49756014	99.0627152	102.54712367	105.95483947
109.23594398	112.26308728	114.99625275	117.51644764	119.87081561
121.98178698	123.78186566	125.31256375	126.66745589	127.84727309
128.51744473	128.53827925	127.81759824	126.52102355	124.81249438
122.90733417	120.8664324	118.67169385	116.26305514	113.52783157
110.73937333	108.03825104	105.26180029	102.32956171	99.16383185
95.36967976	90.77601497	86.73461629	85.38622105	87.08867726
91.10409859	95.54939674	99.47243606	102.75128607	105.58061269
108.09109083	110.4226406	112.62552561	114.73026426	116.60979265
118.30121244	119.96940683	121.67807831	123.31933256	124.64528757
125.4326169	125.66509644	125.36604093	124.69863541	124.03616142
123.5493427	122.5972628	120.9655006	118.75040963	116.1044145
113.13723628	109.85128581	106.42308325	103.01286819	99.75394946
96.3378805	92.84532718	89.26670412	85.4436362	81.52068806
77.46540032	73.15719365	68.69165457	64.11856983	59.15081263
54.40186763	49.97398279	45.05586693	39.93213509	34.82339865
29.40342181	23.86371885	19.73304797	14.83807837	12.88769953
11.09902926	9.45843828	8.15004694	6.63983059	6.22295831
6.11243262	6.04927222	5.99712248	5.9938954	5.98006382
5.94300692	5.88781503	5.7751063	]	

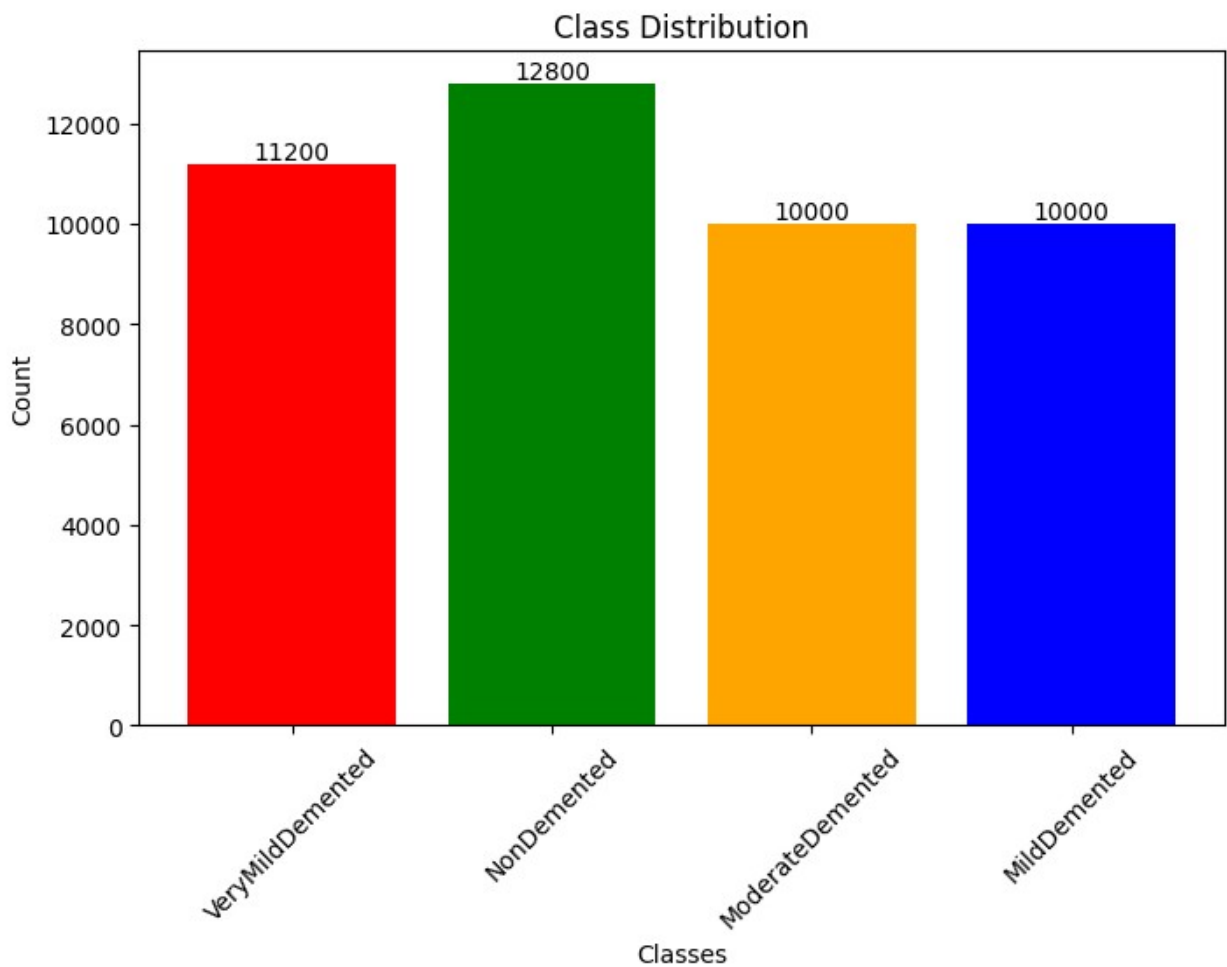
Std Grayscale: [11.85200271 11.94810634 11.98642962 12.01693771
12.05034888 12.13129821

12.32467465	13.35979549	15.80038575	19.42226592	22.16306335
25.82789658				
29.31132131	33.9850534	38.46141666	43.86734328	48.71054299
52.88801212				
57.01114387	60.81675196	63.36038972	66.00408512	67.99189353
69.74373075				
71.59012984	73.24691201	74.89860358	76.31103782	77.72954817
78.75667166				
79.77320591	80.73337051	81.64045616	82.44623818	83.12228022
83.67642718				
84.00112155	84.56193187	84.72914255	85.09263632	85.3993422
85.50207968				
85.80078699	85.49273895	85.5575045	85.11104356	84.76003203
84.0973948				
83.55265122	83.01373848	82.59026007	82.32227154	81.61084703
80.90806335				
79.73930563	78.61987418	77.55997751	76.16350666	74.936011
73.04551638				
71.19245291	68.81601522	66.95608095	66.46109345	67.25707712
69.08553559				
71.22422712	73.25194812	74.71187643	76.336796	77.35514833

78.53680882
79.53520203 80.61129708 81.61447277 82.13455516 82.76779864
83.12444776
83.60063912 84.17117185 84.63490751 85.15215819 85.36380537
85.78752722
85.75535239 86.1201071 85.73607196 85.60148055 85.26222477
84.82721364
84.59238883 83.95200884 83.53476384 82.97091142 82.29944891
81.54628508
80.65713606 79.75149598 78.7534638 77.58719654 76.05292182
74.48377854
72.54608282 70.63734229 68.59370559 66.19122795 63.89738041
60.78877415
56.9493246 52.28953287 47.46964269 41.83954832 37.34084621
31.96215689
28.01348878 24.26246381 20.05766216 17.51535934 13.236047
12.56662036
12.21584498 12.07649667 12.04243514 12.05321578 12.03593274
12.00759738
11.96545328 11.8590386]



```
# Cell 1.7: Class distribution
unique_labels, counts = np.unique(labels_raw, return_counts=True)
plt.figure(figsize=(8, 5))
plt.bar(categories, counts, color=['red', 'green', 'orange', 'blue'])
plt.title('Class Distribution')
plt.xlabel('Classes')
plt.ylabel('Count')
plt.xticks(rotation=45)
for i, count in enumerate(counts):
    plt.text(i, count + 100, str(count), ha='center')
plt.show()
print("Distribution:", dict(zip(categories, counts)))
```



```
Distribution: {'VeryMildDemented': 11200, 'NonDemented': 12800,
'ModerateDemented': 10000, 'MildDemented': 10000}
```

Data Preprocessing

Grayscale load, resize to 128x128 (or 224 for VGG), normalize, post-norm histogram, split, aug preview.

```
# Cell 2.1: Load grayscale, resize, normalize (128x128 for baseline)
images = []
labels = []
```

```
for category in categories:
    path = os.path.join(base_dir, category)
    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name)
        try:
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            img = cv2.resize(img, (128, 128))
            images.append(img)
            labels.append(categories.index(category))
        except Exception as e:
            print(f"Error: {e}")
```

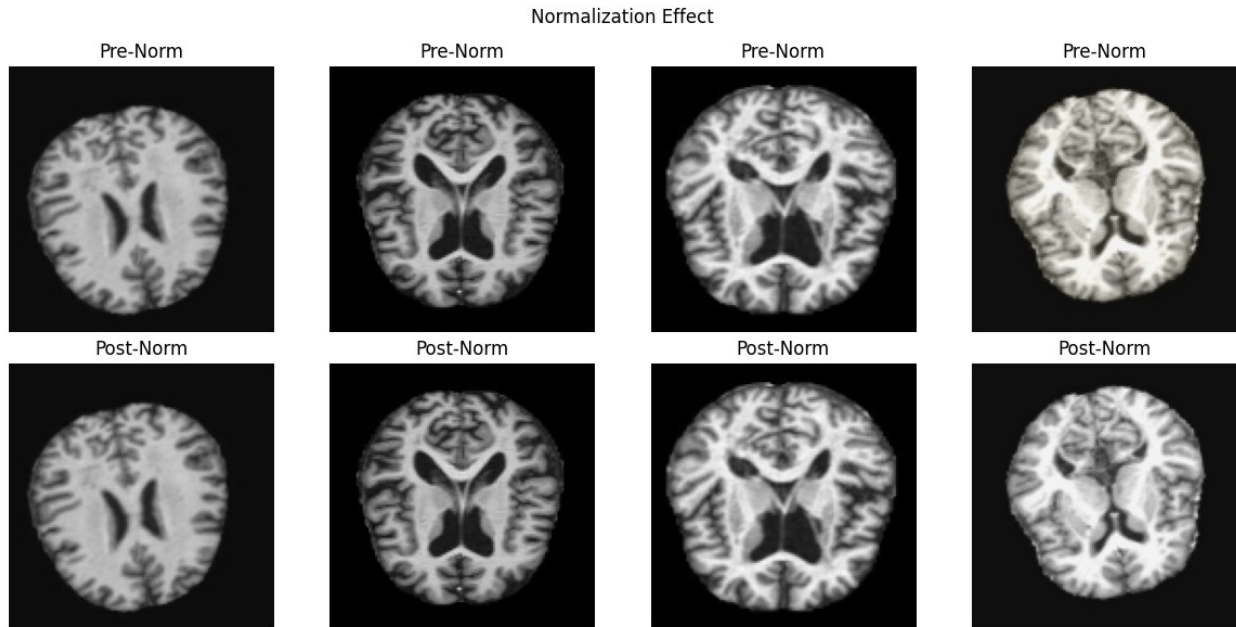
```
images = np.array(images)
labels = np.array(labels)
print(f"Loaded shape: {images.shape}")
```

```
# Channel & normalize
images = np.expand_dims(images, axis=-1)
images = images.astype('float32') / 255.0
print(f"Post-norm shape: {images.shape}, range: [{images.min():.3f}, {images.max():.3f}]")
```

```
Loaded shape: (44000, 128, 128)
Post-norm shape: (44000, 128, 128, 1), range: [0.000, 1.000]
```

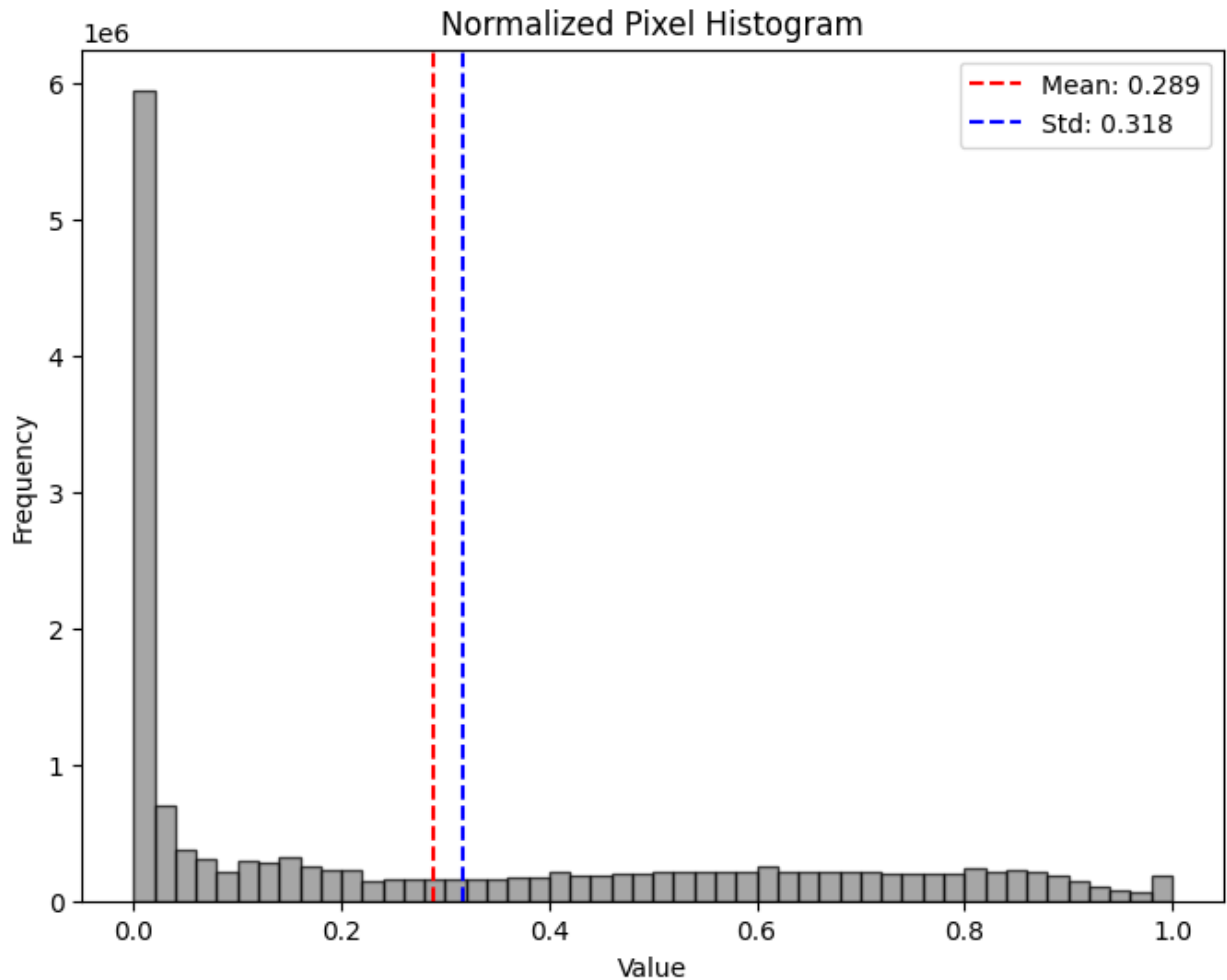
```
# Cell 2.2: Visualize pre/post norm samples
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for i in range(4):
    # Pre-norm (use raw from exploration or reload if needed)
    axes[0, i].imshow(images_raw[i].squeeze() if 'images_raw' in
locals() else images[i].squeeze() * 255, cmap='gray')
    axes[0, i].set_title('Pre-Norm')
    axes[0, i].axis('off')

    axes[1, i].imshow(images[i].squeeze(), cmap='gray', vmin=0,
vmax=1)
    axes[1, i].set_title('Post-Norm')
    axes[1, i].axis('off')
plt.suptitle('Normalization Effect')
plt.tight_layout()
plt.show()
```

```
# Cell 2.3: Post-norm histogram
sample_size = 1000
random_indices = np.random.choice(images.shape[0], sample_size,
replace=False)
sampled_pixels = images[random_indices].ravel()

plt.figure(figsize=(8, 6))
plt.hist(sampled_pixels, bins=50, color='gray', alpha=0.7,
edgecolor='black')
plt.title('Normalized Pixel Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.axvline(np.mean(sampled_pixels), color='red', linestyle='--',
label=f'Mean: {np.mean(sampled_pixels):.3f}')
plt.axvline(np.std(sampled_pixels), color='blue', linestyle='--',
label=f'Std: {np.std(sampled_pixels):.3f}')
plt.legend()
plt.show()
print(f"Mean: {np.mean(sampled_pixels):.3f}, Std:
{np.std(sampled_pixels):.3f}")
```



Mean: 0.289, Std: 0.318

Cell 2.4: Train-test split (fixed: use np.eye for one-hot to avoid TF/protobuf error)

```
from sklearn.model_selection import train_test_split
import numpy as np
```

One-hot with numpy (avoids TF utils issue)

```
y = np.eye(4)[labels]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    images, y, test_size=0.2, stratify=labels, random_state=42
)
```

```
print(f"Train: {X_train.shape[0]}, Test: {X_test.shape[0]}")
```

```
print(f"y_train shape: {y_train.shape}")
```

Train: 35200, Test: 8800

y_train shape: (35200, 4)

Cell 2.5: Augmentation preview

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
sample_img = X_train[0].reshape(1, 128, 128, 1)
aug_iter = datagen.flow(sample_img, batch_size=1)
aug_img = next(aug_iter)[0].reshape(128, 128)
```

```
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(X_train[0].squeeze(), cmap='gray')
plt.title('Original')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(aug_img, cmap='gray')
plt.title('Augmented')
plt.axis('off')
plt.show()
```

2025-11-09 08:40:13.578895: E

```
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
```

```
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
```

```
E0000 00:00:1762677613.806118      48 cuda_dnn.cc:8310] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
```

```
E0000 00:00:1762677613.876057      48 cuda_blas.cc:1418] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
```

[illegible]

```
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'
```

[illegible]

```
AttributeError: 'MessageFactory' object has no attribute  
'GetPrototype'
```

```
-----  
-----  
AttributeError                                Traceback (most recent call  
last)  
AttributeError: 'MessageFactory' object has no attribute  
'GetPrototype'
```

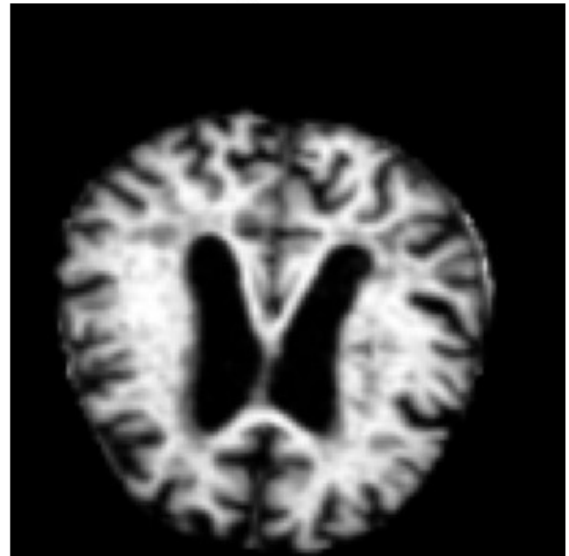
```
-----  
-----  
AttributeError                                Traceback (most recent call  
last)  
AttributeError: 'MessageFactory' object has no attribute  
'GetPrototype'
```

```
-----  
-----  
AttributeError                                Traceback (most recent call  
last)  
AttributeError: 'MessageFactory' object has no attribute  
'GetPrototype'
```

Original



Augmented



Model Building & Training: VGG16 Transfer Learning (Optimized for Kaggle)

Custom CNN underperformed; VGG16 with disk generators, mixed precision, batch=64, no-aug baseline for speed.

```
# Cell 3.1: GPU & Mixed Precision Setup
```

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

import tensorflow as tf
print("TF:", tf.__version__, "CUDA:", tf.test.is_built_with_cuda())
print("GPUs:", tf.config.list_physical_devices('GPU'))
```

```
from tensorflow.keras.mixed_precision import set_global_policy
set_global_policy('mixed_float16')
```

```
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
    tf.config.set_visible_devices(gpus[0], 'GPU')
    print("Logical GPUs:",
tf.config.experimental.list_logical_devices('GPU'))
```

```
with tf.device('/GPU:0'):
    @tf.function
    def test():
        return tf.matmul(tf.random.uniform((1000, 1000)),
tf.random.uniform((1000, 1000)))
    print("GPU test:", test())
```

```
TF: 2.18.0 CUDA: True
```

```
GPUs: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1',
device_type='GPU')]
```

```
Logical GPUs: [LogicalDevice(name='/device:GPU:0', device_type='GPU')]
```

```
GPU test: tf.Tensor(
[[261.4081 250.90381 269.9433 ... 263.105 259.36606 262.8323 ]
 [251.83669 246.49985 262.39294 ... 256.29095 245.39024 258.0463 ]
 [260.36273 253.39073 270.0344 ... 263.9114 250.5151 270.09064]
 ...
 [252.75165 247.77888 264.9352 ... 259.89615 253.89902 254.7935 ]
 [248.93105 240.62117 260.30402 ... 255.00664 247.25449 259.45657]
 [245.51047 239.37677 255.26712 ... 245.34753 240.37527 253.65042]],
shape=(1000, 1000), dtype=float32)
```

```
I0000 00:00:1762677652.142117 48 gpu_device.cc:2022] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 13942 MB
memory: -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0,
compute capability: 7.5
```

```
# Cell 3.2: Disk Generators (224x224, batch=64, no-aug baseline)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.vgg16 import preprocess_input
```

```

base_dir = '/kaggle/input/alzheimers-multiclass-dataset-equal-and-
augmented/combined_images'

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    base_dir,
    target_size=(224, 224),
    color_mode='rgb',
    batch_size=64,
    class_mode='categorical',
    subset='training',
    shuffle=True
)

val_generator = train_datagen.flow_from_directory(
    base_dir,
    target_size=(224, 224),
    color_mode='rgb',
    batch_size=64,
    class_mode='categorical',
    subset='validation'
)

test_datagen =
ImageDataGenerator(preprocessing_function=preprocess_input)
test_generator = test_datagen.flow_from_directory(
    base_dir,
    target_size=(224, 224),
    color_mode='rgb',
    batch_size=64,
    class_mode='categorical',
    shuffle=False
)

print(f"Train batches: {train_generator.samples // 64}")
print("Classes:", train_generator.class_indices)

Found 35200 images belonging to 4 classes.
Found 8800 images belonging to 4 classes.
Found 44000 images belonging to 4 classes.
Train batches: 550
Classes: {'MildDemented': 0, 'ModerateDemented': 1, 'NonDemented': 2,
'VeryMildDemented': 3}

# Cell 3.3: VGG16 Model
from tensorflow.keras.applications import VGG16

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense,
Dropout
from tensorflow.keras.optimizers import Adam

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False

model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax', dtype='float32')
])

model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ————— 0s 0us/step

Model: "sequential"

Layer (type)	Output Shape	
Param #		
vgg16 (Functional)	(None, 7, 7, 512)	
14,714,688		
global_average_pooling2d	(None, 512)	
0 (GlobalAveragePooling2D)		
dense (Dense)	(None, 512)	
262,656		

0	dropout (Dropout)	(None, 512)	
2,052	dense_1 (Dense)	(None, 4)	

Total params: 14,979,396 (57.14 MB)

Trainable params: 264,708 (1.01 MB)

Non-trainable params: 14,714,688 (56.13 MB)

Cell 3.4: Callbacks

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
```

```
callbacks = [
    EarlyStopping(patience=10, restore_best_weights=True,
monitor='val_accuracy'),
    ModelCheckpoint('best_vgg_alzheimer.h5', save_best_only=True,
monitor='val_accuracy'),
    ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-7,
monitor='val_loss')
]
```

Cell 3.5: Train

```
history = model.fit(
    train_generator,
    epochs=50,
    validation_data=val_generator,
    validation_freq=5,
    callbacks=callbacks,
    verbose=1
)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/
data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
self._warn_if_super_not_called()
```

Epoch 1/50

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR


```
I0000 00:00:1762677681.552237      120 service.cc:148] XLA service
0x7e7d5800d220 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
I0000 00:00:1762677681.553164      120 service.cc:156]   StreamExecutor
device (0): Tesla T4, Compute Capability 7.5
I0000 00:00:1762677682.057286      120 cuda_dnn.cc:529] Loaded cuDNN
version 90300
```

```
2/550 _____ 50s 93ms/step - accuracy: 0.2422 - loss:
6.1779
```

```
I0000 00:00:1762677704.694337      120 device_compiler.h:188] Compiled
cluster using XLA! This line is logged at most once for the lifetime
of the process.
```

```
550/550 _____ 134s 199ms/step - accuracy: 0.3954 -
loss: 2.5758 - learning_rate: 1.0000e-04
Epoch 2/50
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/
early_stopping.py:153: UserWarning: Early stopping conditioned on
metric `val_accuracy` which is not available. Available metrics are:
accuracy,loss
```

```
    current = self.get_monitor_value(logs)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/model_chec
kpoint.py:209: UserWarning: Can save best model only with val_accuracy
available, skipping.
```

```
    self._save_model(epoch=epoch, batch=None, logs=logs)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/callback_l
ist.py:145: UserWarning: Learning rate reduction is conditioned on
metric `val_loss` which is not available. Available metrics are:
accuracy,loss,learning_rate.
```

```
    callback.on_epoch_end(epoch, logs)
```

```
550/550 _____ 107s 195ms/step - accuracy: 0.6257 -
loss: 0.8683 - learning_rate: 1.0000e-04
Epoch 3/50
```

```
550/550 _____ 106s 193ms/step - accuracy: 0.6661 -
loss: 0.7547 - learning_rate: 1.0000e-04
Epoch 4/50
```

```
550/550 _____ 106s 193ms/step - accuracy: 0.7081 -
loss: 0.6741 - learning_rate: 1.0000e-04
Epoch 5/50
```

```
550/550 _____ 0s 192ms/step - accuracy: 0.7312 - loss:
0.6263
```

```
WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

```
550/550 _____ 156s 284ms/step - accuracy: 0.7313 -  
loss: 0.6263 - val_accuracy: 0.7830 - val_loss: 0.5412 -  
learning_rate: 1.0000e-04  
Epoch 6/50  
550/550 _____ 108s 196ms/step - accuracy: 0.7582 -  
loss: 0.5737 - learning_rate: 1.0000e-04  
Epoch 7/50  
550/550 _____ 107s 194ms/step - accuracy: 0.7760 -  
loss: 0.5380 - learning_rate: 1.0000e-04  
Epoch 8/50  
550/550 _____ 111s 201ms/step - accuracy: 0.7916 -  
loss: 0.5021 - learning_rate: 1.0000e-04  
Epoch 9/50  
550/550 _____ 114s 206ms/step - accuracy: 0.8053 -  
loss: 0.4677 - learning_rate: 1.0000e-04  
Epoch 10/50  
550/550 _____ 0s 204ms/step - accuracy: 0.8236 - loss:  
0.4389
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 149s 271ms/step - accuracy: 0.8236 -  
loss: 0.4389 - val_accuracy: 0.8436 - val_loss: 0.4063 -  
learning_rate: 1.0000e-04  
Epoch 11/50  
550/550 _____ 109s 199ms/step - accuracy: 0.8381 -  
loss: 0.4100 - learning_rate: 1.0000e-04  
Epoch 12/50  
550/550 _____ 109s 198ms/step - accuracy: 0.8468 -  
loss: 0.3871 - learning_rate: 1.0000e-04  
Epoch 13/50  
550/550 _____ 109s 198ms/step - accuracy: 0.8580 -  
loss: 0.3679 - learning_rate: 1.0000e-04  
Epoch 14/50  
550/550 _____ 108s 195ms/step - accuracy: 0.8646 -  
loss: 0.3471 - learning_rate: 1.0000e-04  
Epoch 15/50  
550/550 _____ 0s 191ms/step - accuracy: 0.8747 - loss:  
0.3338
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 141s 257ms/step - accuracy: 0.8747 -  
loss: 0.3338 - val_accuracy: 0.8800 - val_loss: 0.3163 -  
learning_rate: 1.0000e-04  
Epoch 16/50  
550/550 _____ 107s 193ms/step - accuracy: 0.8841 -  
loss: 0.3083 - learning_rate: 1.0000e-04  
Epoch 17/50  
550/550 _____ 105s 190ms/step - accuracy: 0.8875 -  
loss: 0.2947 - learning_rate: 1.0000e-04  
Epoch 18/50  
550/550 _____ 106s 193ms/step - accuracy: 0.8961 -  
loss: 0.2748 - learning_rate: 1.0000e-04  
Epoch 19/50  
550/550 _____ 105s 190ms/step - accuracy: 0.9051 -  
loss: 0.2628 - learning_rate: 1.0000e-04  
Epoch 20/50  
550/550 _____ 0s 160ms/step - accuracy: 0.9092 - loss:  
0.2514
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 122s 222ms/step - accuracy: 0.9092 -  
loss: 0.2514 - val_accuracy: 0.9098 - val_loss: 0.2505 -  
learning_rate: 1.0000e-04  
Epoch 21/50  
550/550 _____ 104s 189ms/step - accuracy: 0.9109 -  
loss: 0.2419 - learning_rate: 1.0000e-04  
Epoch 22/50  
550/550 _____ 103s 186ms/step - accuracy: 0.9190 -  
loss: 0.2241 - learning_rate: 1.0000e-04  
Epoch 23/50  
550/550 _____ 103s 186ms/step - accuracy: 0.9216 -  
loss: 0.2141 - learning_rate: 1.0000e-04  
Epoch 24/50  
550/550 _____ 105s 190ms/step - accuracy: 0.9269 -  
loss: 0.2029 - learning_rate: 1.0000e-04  
Epoch 25/50  
550/550 _____ 0s 199ms/step - accuracy: 0.9313 - loss:  
0.1968
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 148s 268ms/step - accuracy: 0.9313 -  
loss: 0.1968 - val_accuracy: 0.9157 - val_loss: 0.2224 -  
learning_rate: 1.0000e-04  
Epoch 26/50  
550/550 _____ 110s 199ms/step - accuracy: 0.9373 -  
loss: 0.1830 - learning_rate: 1.0000e-04  
Epoch 27/50  
550/550 _____ 105s 191ms/step - accuracy: 0.9349 -  
loss: 0.1833 - learning_rate: 1.0000e-04  
Epoch 28/50  
550/550 _____ 106s 193ms/step - accuracy: 0.9402 -  
loss: 0.1714 - learning_rate: 1.0000e-04  
Epoch 29/50  
550/550 _____ 106s 192ms/step - accuracy: 0.9438 -  
loss: 0.1614 - learning_rate: 1.0000e-04  
Epoch 30/50  
550/550 _____ 0s 193ms/step - accuracy: 0.9431 - loss:  
0.1587
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 142s 259ms/step - accuracy: 0.9431 -  
loss: 0.1587 - val_accuracy: 0.9375 - val_loss: 0.1781 -  
learning_rate: 1.0000e-04  
Epoch 31/50  
550/550 _____ 104s 189ms/step - accuracy: 0.9476 -  
loss: 0.1488 - learning_rate: 1.0000e-04  
Epoch 32/50  
550/550 _____ 109s 198ms/step - accuracy: 0.9511 -  
loss: 0.1431 - learning_rate: 1.0000e-04  
Epoch 33/50  
550/550 _____ 112s 203ms/step - accuracy: 0.9503 -  
loss: 0.1395 - learning_rate: 1.0000e-04  
Epoch 34/50  
550/550 _____ 111s 202ms/step - accuracy: 0.9545 -  
loss: 0.1325 - learning_rate: 1.0000e-04  
Epoch 35/50  
550/550 _____ 0s 193ms/step - accuracy: 0.9549 - loss:  
0.1322
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 139s 253ms/step - accuracy: 0.9549 -  
loss: 0.1322 - val_accuracy: 0.9406 - val_loss: 0.1596 -  
learning_rate: 1.0000e-04  
Epoch 36/50  
550/550 _____ 227s 414ms/step - accuracy: 0.9584 -  
loss: 0.1225 - learning_rate: 1.0000e-04  
Epoch 37/50  
550/550 _____ 121s 219ms/step - accuracy: 0.9600 -  
loss: 0.1198 - learning_rate: 1.0000e-04  
Epoch 38/50  
550/550 _____ 115s 209ms/step - accuracy: 0.9620 -  
loss: 0.1127 - learning_rate: 1.0000e-04  
Epoch 39/50  
550/550 _____ 115s 209ms/step - accuracy: 0.9607 -  
loss: 0.1146 - learning_rate: 1.0000e-04  
Epoch 40/50  
550/550 _____ 0s 199ms/step - accuracy: 0.9648 - loss:  
0.1057
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 145s 263ms/step - accuracy: 0.9648 -  
loss: 0.1057 - val_accuracy: 0.9476 - val_loss: 0.1406 -  
learning_rate: 1.0000e-04  
Epoch 41/50  
550/550 _____ 92s 167ms/step - accuracy: 0.9666 - loss:  
0.0990 - learning_rate: 1.0000e-04  
Epoch 42/50  
550/550 _____ 112s 204ms/step - accuracy: 0.9674 -  
loss: 0.1010 - learning_rate: 1.0000e-04  
Epoch 43/50  
550/550 _____ 108s 197ms/step - accuracy: 0.9677 -  
loss: 0.0974 - learning_rate: 1.0000e-04  
Epoch 44/50  
550/550 _____ 107s 194ms/step - accuracy: 0.9700 -  
loss: 0.0916 - learning_rate: 1.0000e-04  
Epoch 45/50  
550/550 _____ 0s 166ms/step - accuracy: 0.9707 - loss:  
0.0897
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 127s 231ms/step - accuracy: 0.9707 -  
loss: 0.0897 - val_accuracy: 0.9516 - val_loss: 0.1324 -  
learning_rate: 1.0000e-04  
Epoch 46/50  
550/550 _____ 114s 208ms/step - accuracy: 0.9702 -  
loss: 0.0865 - learning_rate: 1.0000e-04  
Epoch 47/50  
550/550 _____ 109s 197ms/step - accuracy: 0.9710 -  
loss: 0.0880 - learning_rate: 1.0000e-04  
Epoch 48/50  
550/550 _____ 106s 193ms/step - accuracy: 0.9718 -  
loss: 0.0834 - learning_rate: 1.0000e-04  
Epoch 49/50  
550/550 _____ 111s 201ms/step - accuracy: 0.9710 -  
loss: 0.0840 - learning_rate: 1.0000e-04  
Epoch 50/50  
550/550 _____ 0s 207ms/step - accuracy: 0.9739 - loss:  
0.0769
```

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

```
550/550 _____ 149s 271ms/step - accuracy: 0.9739 -  
loss: 0.0769 - val_accuracy: 0.9566 - val_loss: 0.1168 -  
learning_rate: 1.0000e-04
```

Cell 3.6: Plot Curves & Evaluate

```
import matplotlib.pyplot as plt  
from sklearn.metrics import classification_report, confusion_matrix  
import seaborn as sns  
import numpy as np
```

```
plt.figure(figsize=(12, 4))  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Train Acc')  
plt.plot(history.history['val_accuracy'], label='Val Acc')  
plt.title('VGG16 Accuracy')  
plt.legend()  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Val Loss')  
plt.title('VGG16 Loss')  
plt.legend()  
plt.tight_layout()  
plt.show()
```

```
test_generator.reset()
```

```

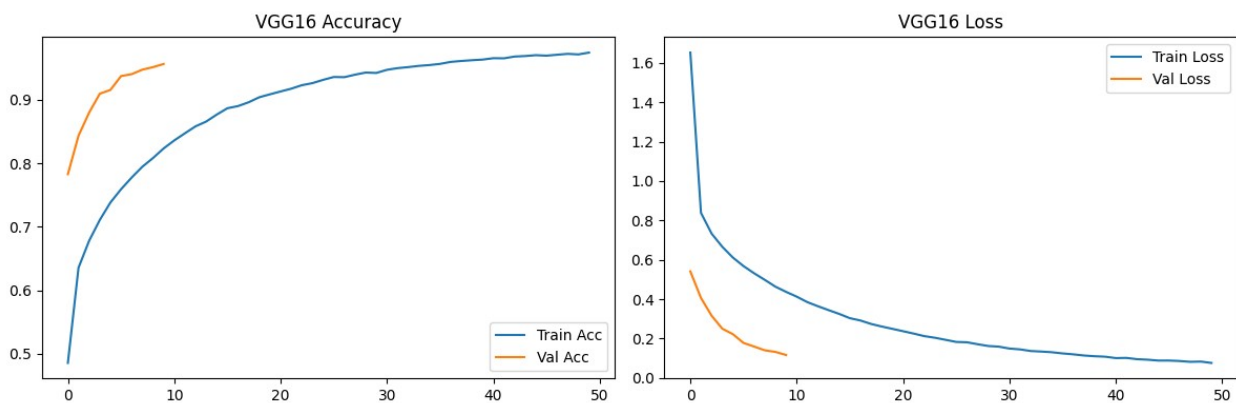
test_loss, test_acc = model.evaluate(test_generator, verbose=0)
print(f"Test Acc: {test_acc:.4f}")

y_pred_prob = model.predict(test_generator)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = test_generator.classes

print("\nClassification Report:")
print(classification_report(y_true, y_pred,
target_names=list(train_generator.class_indices.keys())))

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=list(train_generator.class_indices.keys()),
            yticklabels=list(train_generator.class_indices.keys()))
plt.title('Confusion Matrix')
plt.show()

```



```

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/
data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
    self._warn_if_super_not_called()

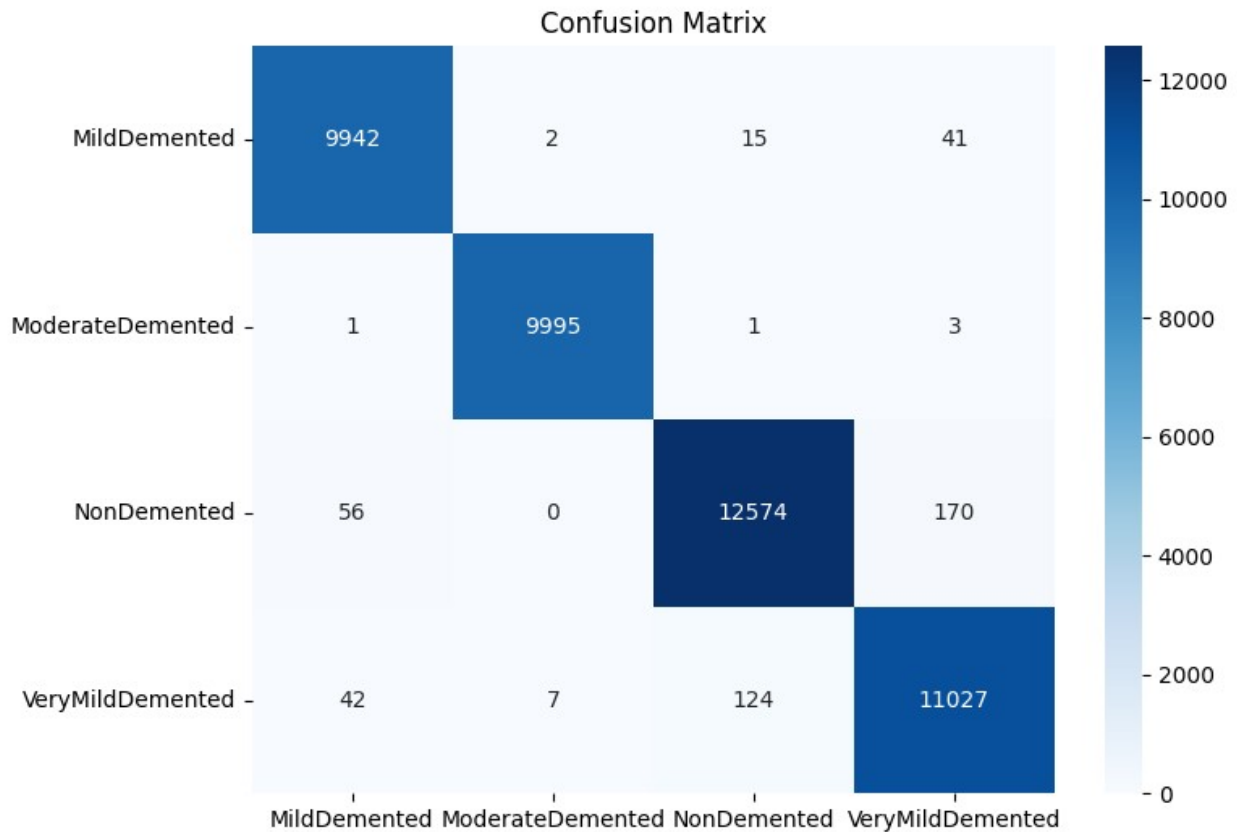
```

Test Acc: 0.9895
688/688 ————— 128s 184ms/step

Classification Report:

	precision	recall	f1-score	support
MildDemented	0.99	0.99	0.99	10000
ModerateDemented	1.00	1.00	1.00	10000
NonDemented	0.99	0.98	0.99	12800
VeryMildDemented	0.98	0.98	0.98	11200

accuracy				0.99	44000
macro avg	0.99	0.99	0.99	0.99	44000
weighted avg	0.99	0.99	0.99	0.99	44000



```
# Cell 4.1: Save Model in Multiple Formats & Download Instructions
# Save as native Keras (.keras) for latest TF (recommended)
model.save('alzheimer_vgg16.keras')
```

```
# Or legacy HDF5 (.h5) for compatibility
model.save('alzheimer_vgg16.h5')
```

```
# Verify save
print("Model saved! Sizes:")
!ls -lh *.keras *.h5
```

```
# What: Exports in .keras (modern, smaller) or .h5 (legacy).
# Why: .keras is TF 2.13+ default; .h5 for older tools. ~50-100MB each.
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras


```
format, e.g. `model.save('my_model.keras')` or  
`keras.saving.save_model(model, 'my_model.keras')`.
```

Model saved! Sizes:

```
-rw-r--r-- 1 root root 60M Nov  9 10:27 alzheimer_vgg16.h5  
-rw-r--r-- 1 root root 62M Nov  9 10:27 alzheimer_vgg16.keras  
-rw-r--r-- 1 root root 60M Nov  9 10:19 best_vgg_alzheimer.h5
```

Cell: List & Zip Models for Download

```
import os  
import zipfile
```

List files

```
models_dir = '/kaggle/working/'  
models = ['alzheimer_vgg16.h5', 'alzheimer_vgg16.keras',  
          'best_vgg_alzheimer.h5']  
print("Models in /kaggle/working:")  
for m in models:  
    path = os.path.join(models_dir, m)  
    size = os.path.getsize(path) if os.path.exists(path) else 0  
    print(f"- {m}: {size / (1024**2):.1f} MB")
```

Zip all for bulk download

```
zip_path = '/kaggle/working/alzheimer_models.zip'  
with zipfile.ZipFile(zip_path, 'w') as zf:  
    for m in models:  
        zf.write(os.path.join(models_dir, m), m)  
print(f"\nZipped: {zip_path} – Download from Output panel!")
```

```
Models in /kaggle/working/:  
- alzheimer_vgg16.h5: 59.2 MB  
- alzheimer_vgg16.keras: 61.3 MB  
- best_vgg_alzheimer.h5: 59.2 MB
```

```
Zipped: /kaggle/working/alzheimer_models.zip – Download from Output  
panel!
```